Compléments d'informatique

Projet 2 : simulateur de collisions de particules

L'objectif concret de ce projet est d'implémenter un simulateur de particules se déplaçant dans une boîte rectangulaire et entrant en collision les unes avec les autres ainsi qu'avec les parois de la boîte. L'objectif pédagogique est de vous faire pratiquer la programmation modulaire, le programme complet étant constitué de plusieurs modules séparés interagissant entre eux, ainsi que de vous faire réfléchir à l'efficacité algorithmique de votre implémentation. Une fois le simulateur implémenté, vous l'utiliserez pour vérifier expérimentalement une loi classique de la physique statistique : la distribution de Maxwell-Boltzmann des vitesses dans un gaz à l'équilibre. Ce projet est à réaliser **seul ou à deux**. La date limite de remise est précisée sur eCampus.

1 Description du simulateur

Le simulateur modélise un gaz bidimensionnel constitué de particules assimilées à des disques rigides. Les particules se déplacent en ligne droite à vitesse constante tant qu'elles ne rencontrent rien, et rebondissent de manière élastique (c'est-à-dire sans perte d'énergie) lorsqu'elles heurtent une paroi ou une autre particule.

Représentation de l'environnement. Les particules évoluent dans une boîte rectangulaire $[0, W] \times [0, H]$, avec W la largeur de la boîte et H sa hauteur. Les parois verticales de la boîte sont situées en $x = 0$ et $x = W$, les parois horizontales en $y = 0$ et $y = H$.

Particules. Chaque particule i est caractérisée par :

- son rayon $r_i > 0$ et sa masse $m_i > 0$;
- sa position $\mathbf{p}_i = (x_i, y_i)$, c'est-à-dire la position de son centre ;
- sa vitesse $\mathbf{v}_i = (v_{x,i}, v_{y,i})$;
- une couleur, utilisée uniquement pour l'affichage ;

L'énergie cinétique d'une particule est donnée par $E_i = \frac{1}{2} m_i \|\mathbf{v}_i\|^2$. Les collisions étant élastiques, l'énergie cinétique totale du système est conservée au cours de la simulation.

Mouvement libre. En l'absence de collision, une particule se déplace en ligne droite à vitesse constante. Après un intervalle de temps Δt, sa position devient :

$$\mathbf{p}_i \leftarrow \mathbf{p}_i + \Delta t\, \mathbf{v}_i.$$

Collision avec une paroi. Une particule touche la paroi verticale de droite lorsque $x_i + r_i = W$ et celle de gauche lorsque $x_i - r_i = 0$ (et de même pour les parois horizontales avec y_i et H). Le temps (relatif au temps courant) avant que la particule i ne touche une paroi verticale est donné par :

$$\Delta t_x = \begin{cases} (W - r_i - x_i)/v_{x,i} & \text{si } v_{x,i} > 0, \\ (r_i - x_i)/v_{x,i} & \text{si } v_{x,i} < 0, \\ +\infty & \text{si } v_{x,i} = 0. \end{cases}$$

Le temps Δt_y avant une collision avec une paroi horizontale se calcule de la même manière en remplaçant x par y et W par H. Lors d'une collision avec une paroi verticale, la composante horizontale de la vitesse est simplement inversée ($v_{x,i} \leftarrow -v_{x,i}$). Lors d'une collision avec une paroi horizontale, c'est la composante verticale qui est inversée ($v_{y,i} \leftarrow -v_{y,i}$).

Collision entre deux particules. Soient deux particules i et j. Notons $\Delta\mathbf{p} = \mathbf{p}_j - \mathbf{p}_i$, $\Delta\mathbf{v} = \mathbf{v}_j - \mathbf{v}_i$ et $\sigma = r_i + r_j$. Si les deux particules continuent à se déplacer en ligne droite, elles se touchent au temps (relatif) Δt tel que $\|\Delta\mathbf{p} + \Delta t\, \Delta\mathbf{v}\| = \sigma$, c'est-à-dire solution de l'équation du second degré :

$$(\Delta\mathbf{v} \cdot \Delta\mathbf{v})\, \Delta t^2 + 2(\Delta\mathbf{v} \cdot \Delta\mathbf{p})\, \Delta t + (\Delta\mathbf{p} \cdot \Delta\mathbf{p} - \sigma^2) = 0,$$

où $\cdot$ désigne le produit scalaire. On en déduit le temps avant collision :

$$\Delta t = \begin{cases} +\infty & \text{si } \Delta\mathbf{v} \cdot \Delta\mathbf{p} \geq 0 \text{ (les particules s'éloignent l'une de l'autre)}, \\ +\infty & \text{si } d < 0 \text{ (les trajectoires ne se croisent pas)}, \\ -\dfrac{\Delta\mathbf{v} \cdot \Delta\mathbf{p} + \sqrt{d}}{\Delta\mathbf{v} \cdot \Delta\mathbf{v}} & \text{sinon}, \end{cases}$$

avec $d = (\Delta\mathbf{v} \cdot \Delta\mathbf{p})^2 - (\Delta\mathbf{v} \cdot \Delta\mathbf{v})(\Delta\mathbf{p} \cdot \Delta\mathbf{p} - \sigma^2)$.

Au moment de la collision, les vitesses des deux particules sont modifiées selon les lois du choc élastique entre deux disques (conservation de la quantité de mouvement et de l'énergie cinétique, la force d'impact s'exerçant selon la droite reliant les deux centres) :[1]

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \frac{2m_j}{m_i + m_j} \frac{\Delta\mathbf{v} \cdot \Delta\mathbf{p}}{\|\Delta\mathbf{p}\|^2} \Delta\mathbf{p},$$
$$\mathbf{v}_j \leftarrow \mathbf{v}_j - \frac{2m_i}{m_i + m_j} \frac{\Delta\mathbf{v} \cdot \Delta\mathbf{p}}{\|\Delta\mathbf{p}\|^2} \Delta\mathbf{p}.$$

1. Voir `https://en.wikipedia.org/wiki/Elastic_collision` ou vos cours de mécanique pour la dérivation détaillée de ces équations.

Simulation dirigée par les événements. Une approche naïve pour simuler ce système consisterait à faire avancer toutes les particules par petits pas de temps fixes et à vérifier à chaque pas si des particules se chevauchent. Cette approche est à la fois coûteuse et imprécise (des collisions peuvent être manquées si le pas de temps est trop grand). Nous utiliserons plutôt une *simulation dirigée par les événements* (*event-driven simulation*). Un *événement* est une collision d'une particule avec une paroi verticale, avec une paroi horizontale ou avec une autre particule. Il est caractérisé par son type, la ou les particules impliquées et le temps (absolu) auquel il se produit.

Entre deux événements, toutes les particules se déplacent en ligne droite. Il suffit donc de déterminer le prochain événement, c'est-à-dire celui qui se produira le plus tôt parmi toutes les collisions possibles (avec les parois pour chaque particule et entre chaque paire de particules), de déplacer toutes les particules jusqu'au moment de cet événement, de mettre à jour la vitesse de la ou des particules impliquées, puis de recommencer. Plus précisément, faire avancer la simulation du temps courant t jusqu'au temps $t + \Delta t$ consiste à appliquer l'algorithme suivant :

1. Déterminer le prochain événement e et son temps t_e.
2. Tant que $t_e < t + \Delta t$:
   (a) Déplacer toutes les particules en ligne droite pendant une durée $t_e - t_c$, où t_c est le temps courant de la simulation.
   (b) Mettre le temps courant de la simulation à t_e.
   (c) Appliquer l'événement e, c'est-à-dire mettre à jour la vitesse de la ou des particules impliquées.
   (d) Déterminer le nouveau prochain événement e et son temps t_e.
3. Déplacer toutes les particules en ligne droite jusqu'au temps $t + \Delta t$, qui devient le nouveau temps courant de la simulation.

Notez que l'intervalle Δt ne sert qu'à déterminer les instants auxquels on observe la simulation (par exemple pour générer les images de l'animation). Il n'a aucune influence sur la précision de la simulation puisque tous les événements survenant dans l'intervalle sont traités exactement.

2 Implémentation

L'implémentation du simulateur est basée sur plusieurs modules, dont deux sont à implémenter par vos soins :

— `particle.c/.h` implémentant une particule.
— `simulator.c/.h` implémentant le simulateur.

Nous vous fournissons les fichiers d'entête `particle.h` et `simulator.h`, ainsi que les fichiers suivants, qui ne doivent pas être modifiés :

— `vector.c/.h` définissant un type (non opaque) `Vector` représentant un vecteur à deux dimensions ainsi que quelques opérations de base sur ces vecteurs (somme, différence, multiplication par un scalaire, produit scalaire et norme).
— `event.c/.h` définissant un type opaque `Event` représentant un événement (voir plus bas).
— `color.h` définissant le type `Color` utilisé pour représenter la couleur d'une particule.
— `main.c` vous permettant de tester votre code (voir plus bas).
— `gifenc.c/.h` et `simulation_gif.c/.h` permettant de générer une animation de la simulation au format GIF.
— un `Makefile` permettant de compiler le projet.
— un répertoire `Examples` contenant plusieurs fichiers décrivant des configurations initiales de particules pour vos tests.

Certains de ces fichiers et ce qu'ils contiennent, ou devront contenir, sont décrits brièvement ci-dessous. Nous vous renvoyons aux déclarations contenues dans les différents fichiers d'entête pour les détails.

2.1 Fichiers `event.c` et `event.h`

Ces fichiers qui vous sont fournis définissent le type opaque `Event` représentant un événement. Un événement est caractérisé par :

— son type, de type `EventType`, qui peut valoir `XWALL` (collision avec une paroi verticale, qui modifie donc la composante x de la vitesse), `YWALL` (collision avec une paroi horizontale), `COLLISION` (collision entre deux particules) ou `NONE` (aucun événement) ;
— les particules impliquées (la seconde particule valant `NULL` pour une collision avec une paroi) ;
— le temps *absolu* auquel l'événement se produit.

Un événement est créé par la fonction `eventCreate` et libéré par `eventFree`. La fonction `eventSet` permet de modifier tous les champs d'un événement existant et les fonctions `eventGetType`, `eventGetParticle1`, `eventGetParticle2` et `eventGetTime` d'accéder à ses champs.

2.2 Fichiers `particle.c` et `particle.h`

Ces fichiers définissent le type opaque `Particle` représentant une particule et les fonctions associées. Vous devez définir la structure `Particle_t` dans `particle.c` et implémenter toutes les fonctions déclarées dans `particle.h`, à savoir :

— `particleCreate` et `particleFree`, qui créent et libèrent une particule.
— Les accesseurs `particleGetRadius`, `particleGetMass`, `particleGetPosition`, `particleGetVelocity`, `particleGetColor`.

— `particleGetNumCollisions`, qui renvoie le nombre de collisions qu'une particule a subies avec d'autres particules depuis le début de la simulation.
— `particleKineticEnergy`, qui renvoie l'énergie cinétique de la particule.
— `particleMove`, qui déplace la particule en ligne droite pendant une durée `dt`.
— `particleGetXWallCollisionTime` et `particleGetYWallCollisionTime`, qui renvoient le temps (relatif au temps courant) avant que la particule ne touche respectivement une paroi verticale et une paroi horizontale, ou `INFINITY_TIME` si cela n'arrivera jamais.
— `particleGetCollisionTime`, qui renvoie le temps (relatif au temps courant) avant que les deux particules en argument n'entrent en collision, ou `INFINITY_TIME` si elles n'entreront jamais en collision.
— Les fonctions `particleHandleXWallCollision`, `particleHandleYWallCollision` et `particleHandleCollision`, qui mettent à jour la vitesse de la ou des particules suite à une collision avec une paroi verticale, avec une paroi horizontale ou entre deux particules.
— `particleOverlap`, qui renvoie `true` si les deux particules en argument se chevauchent ou se touchent (la distance entre leurs centres est inférieure ou égale à la somme de leurs rayons).
— `particleOutside`, qui renvoie `true` si la particule touche ou dépasse l'une des parois d'une boîte de dimensions `width`×`height`.

La constante `INFINITY_TIME`, définie dans `particle.h`, représente un temps infini. Toutes les formules nécessaires sont données à la section 1.

2.3 Fichiers `simulator.c` et `simulator.h`

Ces fichiers définissent le type opaque `Simulator` utilisé pour gérer une simulation et les fonctions associées. Vous devez définir la structure `Simulator_t` dans `simulator.c` et implémenter toutes les fonctions déclarées dans `simulator.h`, à savoir :

— `simulatorCreate`, qui crée un simulateur pour une boîte de dimensions `width`×`height` pouvant contenir au plus `maxNumParticles` particules, et `simulatorFree`, qui libère le simulateur ainsi que toutes les particules qu'il contient.
— `simulatorAddParticle`, qui ajoute une particule à la simulation. La particule ne doit pas être ajoutée (et la fonction doit renvoyer `false`) si le nombre maximum de particules est atteint, si la particule touche ou dépasse une paroi de la boîte, si elle chevauche une particule déjà présente, ou si la simulation a déjà commencé (c'est-à-dire si `simulatorStep` ou `simulatorGetNextEvent` a déjà été appelée). Sinon, la particule est ajoutée, la fonction renvoie `true` et le simulateur devient responsable de la libération de la particule.
— Les accesseurs `simulatorGetWidth`, `simulatorGetHeight`, `simulatorGetElapsedTime` (le temps courant de la simulation, initialement 0), `simulatorGetNumberOfParticles` et `simulatorGetNumCollisions` (le nombre total de collisions entre particules depuis

le début de la simulation, les collisions avec les parois n'étant pas comptabilisées).
— `simulatorGetParticle`, qui renvoie la particule d'indice `index` (les particules étant indicées de 0 à $N-1$ dans l'ordre de leur ajout) ou `NULL` si l'indice n'est pas valide.
— `simulatorGetNextEvent`, qui renvoie le prochain événement qui se produira dans la simulation, c'est-à-dire l'événement de temps minimum parmi toutes les collisions possibles. Si aucun événement ne peut se produire, un événement de type `NONE` et de temps `INFINITY_TIME` est renvoyé. L'événement renvoyé appartient au simulateur et ne doit pas être libéré par l'appelant.
— `simulatorStep`, qui fait avancer la simulation d'une durée `dt` selon l'algorithme décrit à la section 1.
— `simulatorStatParticles`, qui renvoie la moyenne sur toutes les particules de la valeur renvoyée par la fonction `f` passée en argument.

2.4 Fichier `main.c`

Un fichier `main.c` vous est fourni pour tester votre implémentation. Via le fichier `Makefile` fourni également, ce fichier `main.c` crée un exécutable `simulate` qui peut être utilisé pour effectuer une simulation. La commande suivante :

```
./simulate -f F -s S -t DT -g G -o O
```

effectuera une simulation de `S` pas de temps de durée `DT` chacun, où :
— `F` est un fichier décrivant la configuration initiale des particules (voir plus bas) ;
— `G` est la taille maximale (en pixels) de l'animation générée ;
— `O` est le nom du fichier GIF généré.
Si l'option `-f` n'est pas spécifiée, le programme génère aléatoirement `N` particules dans une boîte de taille 30×30, où `N` est spécifié par l'option `-n N`. Les valeurs par défaut des différents paramètres peuvent être obtenues par la commande `./simulate -h`. Le programme affiche l'énergie cinétique moyenne des particules au début et à la fin de la simulation (qui doivent être égales, aux erreurs d'arrondi près), le nombre total de collisions entre particules et le nombre moyen de collisions par particule.

Le répertoire `Examples` contient des exemples de fichiers de configuration pour vos tests. Ces fichiers ont le format suivant : la première ligne contient la largeur W et la hauteur H de la boîte ainsi que le nombre N de particules. Chacune des N lignes suivantes décrit une particule par sa position (x, y), sa vitesse (v_x, v_y), sa masse, son rayon et sa couleur (un entier entre 1 et 6). Par exemple, la commande

```
./simulate -f Examples/explosion.txt -s 300 -o explosion.gif
```

génère une animation de 300 images de l'expansion d'un nuage de particules initialement concentrées au centre de la boîte.

3 Conseils d'implémentation

La modularité du code fait que vous pouvez implémenter et tester `particle.c` indépendamment de `simulator.c`. Nous vous conseillons de commencer par `particle.c` et de tester soigneusement chacune des fonctions (en particulier le calcul des temps de collision et la mise à jour des vitesses) sur des petits exemples dont vous pouvez calculer le résultat à la main, avant de passer à `simulator.c`.

Dans `simulator.c`, la fonction la plus importante est `simulatorGetNextEvent`. Dans sa version la plus simple, elle calcule, pour chaque particule, le temps avant sa prochaine collision avec chacune des parois et avec chacune des autres particules, et renvoie l'événement correspondant au temps minimum. Attention : les fonctions de `particle.h` renvoient des temps relatifs au temps courant, alors que les événements contiennent des temps absolus. Lors de l'implémentation de `simulatorStep`, vous devrez veiller à libérer correctement la mémoire allouée pour les événements.

Vers une implémentation plus efficace. Avec l'implémentation simple décrite ci-dessus, la recherche du prochain événement par la fonction `simulatorGetNextEvent` a une complexité en $\Theta(N^2)$, où N est le nombre de particules, puisque toutes les paires de particules sont considérées. Cette recherche étant effectuée après chaque événement, et le nombre d'événements par unité de temps étant lui-même proportionnel à N, le temps de calcul d'une simulation croît très rapidement avec le nombre de particules. Une implémentation correcte de cette version simple est suffisante pour obtenir tous les points sur l'implémentation, mais nous vous encourageons à réfléchir à une solution plus efficace.

Une observation clé vers une implémentation plus efficace est qu'un événement ne modifie la vitesse que d'une ou deux particules. Les prochains événements des autres particules restent donc pour la plupart inchangés, et il est inutile de tout recalculer après chaque événement. En mémorisant, pour chaque particule, son prochain événement, on peut, après avoir appliqué un événement, se contenter de recalculer les prochains événements des particules impliquées dans cet événement, ainsi que ceux des particules dont le prochain événement mémorisé impliquait l'une de ces particules (et qui n'est donc plus valide). À titre indicatif, pour une simulation de 500 particules, une telle implémentation est environ 50 fois plus rapide que l'implémentation simple.

D'autres améliorations sont possibles, par exemple en utilisant une file à priorité pour stocker les événements ou en découpant la boîte en cellules pour ne prendre en compte que les collisions entre particules proches. Cependant, ces améliorations dépassent le cadre de ce cours et nous ne vous encourageons pas à les explorer.

4 Application : distribution des vitesses à l'équilibre

En physique statistique, la *distribution de Maxwell-Boltzmann* décrit la distribution des vitesses des particules d'un gaz à l'équilibre thermique. Quelle que soit la distribution initiale des vitesses, les collisions successives entre particules redistribuent l'énergie entre celles-ci, et la distribution des vitesses converge vers cette distribution d'équilibre. Dans le cas d'un gaz bidimensionnel constitué de particules de même masse m, la densité de probabilité de la norme $v = \|\mathbf{v}\|$ de la vitesse d'une particule est donnée à l'équilibre par :

$$f(v) = \frac{mv}{k_B T} \exp\left(-\frac{mv^2}{2k_B T}\right), \quad v \geq 0,$$

où k_B est la constante de Boltzmann et T la température du gaz. En deux dimensions, l'énergie cinétique moyenne d'une particule est égale à $k_B T$. On a donc $k_B T = \bar{E}$, où $\bar{E}$ est l'énergie cinétique moyenne des particules, qui est conservée au cours de la simulation et peut être calculée par `simulatorStatParticles(s, particleKineticEnergy)`.

On vous demande de vérifier expérimentalement cette loi à l'aide de votre simulateur. Pour ce faire, écrivez un programme `maxwell.c` (contenant sa propre fonction `main` et n'utilisant que les fonctions déclarées dans `particle.h` et `simulator.h`) qui :

1. crée un simulateur contenant un grand nombre de particules (par exemple $N = 500$ particules de rayon 1 et de masse 1 dans une boîte de taille 100×100), placées aléatoirement dans la boîte, ayant *toutes la même norme de vitesse* (par exemple 1) mais une direction tirée aléatoirement de manière uniforme. La distribution initiale des vitesses est donc très éloignée de la distribution de Maxwell-Boltzmann ;
2. fait avancer la simulation jusqu'à ce que chaque particule ait subi en moyenne un nombre suffisant de collisions (par exemple au moins 50, ce que vous pouvez vérifier grâce à `simulatorStatParticles(s, particleGetNumCollisions)`) ;
3. écrit dans un fichier la norme de la vitesse de chacune des particules.

À partir de ce fichier, générez une figure `histogram.pdf` représentant l'histogramme (normalisé, de manière à approximer une densité de probabilité) des normes des vitesses à la fin de la simulation, superposé à la courbe théorique $f(v)$ ci-dessus. Vous pouvez utiliser l'outil de votre choix pour générer cette figure (Python et matplotlib, un tableur, etc.). Nous vous conseillons de vérifier également comment l'histogramme évolue au cours du temps, en le traçant à plusieurs instants de la simulation. Pour obtenir un histogramme plus lisse, vous pouvez augmenter le nombre de particules (si votre implémentation est suffisamment efficace).

Pour compiler votre programme, nous utiliserons la commande `make maxwell` prévue à cet effet dans le fichier `Makefile`.

5 Soumission

Vous devez soumettre (uniquement) les fichiers suivants sur Gradescope :

- `particle.c` et `simulator.c`.
- `maxwell.c`
- `histogram.pdf`
- `rapport.txt`

Votre code doit compiler sans erreur ni avertissement avec les options de compilation spécifiées dans le `Makefile` fourni. Dans `rapport.txt` vous sont demandés les contributions de chacun au projet, ainsi qu'une brève description de votre implémentation de `simulatorGetNextEvent` et `simulatorStep` si vous avez implémenté une version plus efficace que la version simple. Par la soumission de ce fichier vous attestez également avoir respecté le règlement en matière de plagiat du cours. Le fichier `histogram.pdf` devra contenir la figure décrite dans la section précédente.

Bon travail !